

CEN3024C Module 8 Project

For this project we will enhance the TaskManager application by implementing automated testing and continuous integration (CI) using GitHub Actions. Building on previous work, we will create and verify unit tests, then automate the build and test process through a workflow file in the repository.

1. Create Task Tests:

- Use the UserRepository and UserService test examples in the GitHub Classroom repository as references.
- Implement new tests for the TaskRepository and TaskService classes.
- For TaskRepository: create both an in-memory test class and a mocking test class.
- For TaskService: create one mocking test class.

2. Verify Tests

- Right-click on the java subfolder under the test directory in IntelliJ and select Run all tests.
- Ensure all tests display green checkmarks indicating success.
- Take a snip of the test results for submission

3. Create a GitHub Actions Workflow

- Create a .github/workflows folder, then create a new file named build-and-test.yml.

Sample Workflow Content

name: Build and Run TaskManager Application

on:

push:

paths:

- '/*.java'**

pull_request:

paths:

- '/*.java'**

workflow_dispatch: # Enables manual trigger

jobs:

test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3

- name: Set up JDK 17

uses: actions/setup-java@v3

with:

java-version: '17'

distribution: 'temurin'

- name: Run Maven Tests

run: mvn -f pom.xml test

build-and-run:

```

needs: test
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v3

  - name: Set up JDK 17
    uses: actions/setup-java@v3
    with:
      java-version: '17'
      distribution: 'temurin'

  - name: Build with Maven
    run: mvn -f pom.xml clean package -DskipTests

  - name: Run Spring Boot Application
    run: |
      java -jar target/*.jar &
      echo $! > springboot.pid
    timeout-minutes: 5

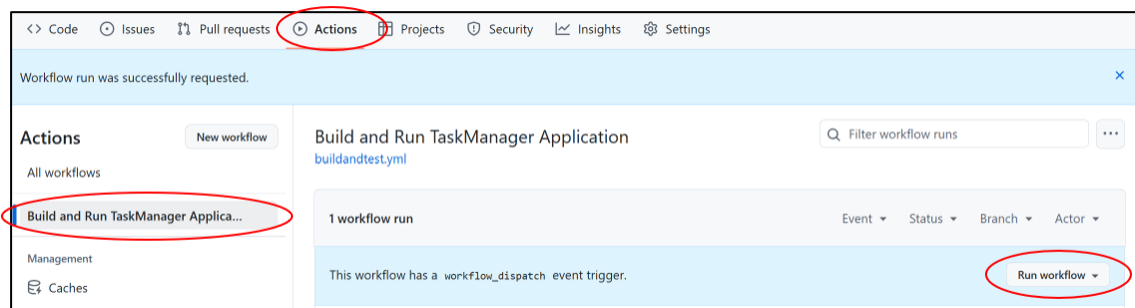
  - name: Shutdown Spring Boot Application
    run: kill $(cat springboot.pid)

```

- The workflow should automatically build and test the application when a .java file is added or modified.

4. Workflow Verification

- Commit and push your .yaml workflow file.
- Push a change (this can be as simple as adding or changing a comment in a .java file).
- Navigate to the **Actions** tab in GitHub to verify the automated build and test process.
- Demonstrate a successful test run (either automatic or manual) using a screenshot of the executed workflow



- From the Actions tab you can view your workflow run summaries:

The screenshot shows the 'Build and Run TaskManager Application' workflow page. At the top, it says '3 workflow runs'. Below this, there's a blue banner indicating the workflow has a 'workflow_dispatch' event trigger. A 'Run workflow' button is visible. The list of runs includes:

- comment #2**: Build and Run TaskManager Application #3: Commit 2cc819f pushed by ProfSingletary. Status: Success (green checkmark). Duration: 1m 2s. Triggered 1 minute ago.
- Build and Run TaskManager Application**: Build and Run TaskManager Application #2: Manually run by ProfSingletary. Status: Success (green checkmark). Duration: 1m 12s. Triggered 12 minutes ago.
- Build and Run TaskManager Application**: Build and Run TaskManager Application #1: Manually run by ProfSingletary. Status: Failure (red X). Duration: 11s. Triggered 19 minutes ago.

- Select a specific run instance to drill down on the results (a green checkmark means success)

The screenshot shows the detailed view for the successful run 'Build and Run TaskManager Application #2'. It includes a summary section with the following details:

- Status**: Success
- Total duration**: 1m 12s
- Artifacts**: None
- Triggered by**: ProfSingletary
- Triggered at**: 7fb12ba
- Branch**: main

Below the summary, there's a section for 'buildandtest.yml' triggered by 'workflow_dispatch'. It shows a job graph with two jobs: 'test' (33s) and 'build-and-run' (30s), both marked as successful with green checkmarks.

- You can find your build and test results by selecting the applicable job button in the Action. The output is very lengthy, but if you scroll to the bottom of the test output here is what a successful test run looks like:

```

9095 [INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.206 s -- in
edu.fscj.cen3024c.taskmanager.repository.UserRepositoryInMemoryTest
9096 [INFO] Running edu.fscj.cen3024c.taskmanager.repository.TaskRepositoryMockTest
9097 [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.449 s -- in
edu.fscj.cen3024c.taskmanager.repository.TaskRepositoryMockTest
9098 [INFO] Running edu.fscj.cen3024c.taskmanager.service.UserServiceTest
9099 [INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.167 s -- in
edu.fscj.cen3024c.taskmanager.service.UserServiceTest
9100 [INFO] Running edu.fscj.cen3024c.taskmanager.service.TaskServiceTest
9101 [INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.382 s -- in
edu.fscj.cen3024c.taskmanager.service.TaskServiceTest
9102 [INFO] Running edu.fscj.cen3024c.taskmanager.service.TaskServiceTest
9103 [INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.042 s -- in
edu.fscj.cen3024c.taskmanager.service.TaskServiceTest
9104 [INFO]

```

Submission

Submit your project and screensnips showing your test results to the GitHub classroom repo.